



Programação Orientada a Objetos

Polimorfismo

Maurício da Motta Braga
mmb2@cesar.school





"With polymorphism, you can write code that doesn't care what type of object it is dealing with — as long as it understands the message you send it."

Bjarne Stroustrup.

P OO – quatro pilares

Abstração

Encapsulamento

Herança

Polimorfismo



P OO – quatro pilares

Abstração

Encapsulamento

Herança

Polimorfismo

Polimorfismo

Recurso fundamental da programação orientada a objetos para construção de softwares modernos.

Permite que um objeto possa ser utilizado (acessado / referenciado) de várias formas dentro de um software.

Através dele, objetos poderão interagir com um outro objeto sem saber a sua forma específica.

Polimorfismo

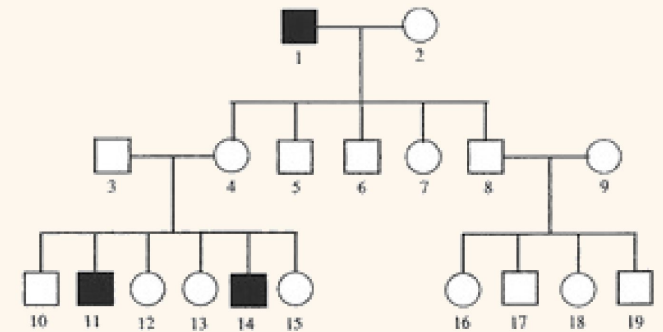
Viabiliza que um mesmo nome possa representar diferentes comportamentos, selecionados através de um mecanismo automático.

Permite a uma mesma ação ser implementada de diversas maneiras em objetos diferentes.

Com o polimorfismo, métodos com o mesmo nome podem apresentar comportamentos diferentes, dependendo do tipo do objeto utilizado.

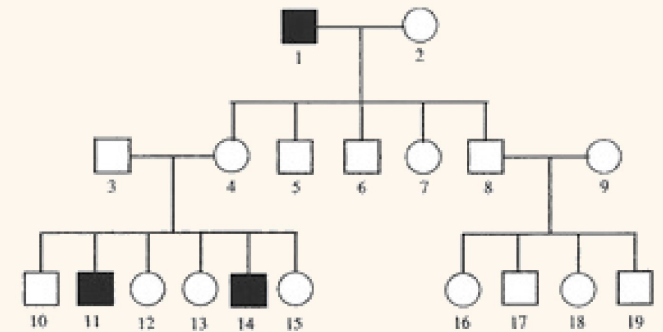
Polimorfismo

- ▶ Conjunto de características e de recursos das linguagens OO que permitem:
 - ▶ A definição de comportamentos e/ou formas diferentes para um mesmo “conceito de ação” (sobrecarga de método), em uma mesma hierarquia de classes;
 - ▶ A visão de um mesmo tipo sob a forma de vários outros tipos de seus ancestrais;
 - ▶ O uso de um método com diferentes implementações em vários tipos diferentes (também em uma mesma hierarquia de classes) através da sobrescrita.



Sobrecarga de métodos

- ▶ É possível ter métodos que possuem o mesmo nome dentro de uma mesma classe.
- ▶ Quando isso ocorre, esses métodos são ditos serem “sobrecarregados”.
- ▶ Para fins práticos, métodos sobrecarregados são métodos diferentes que apenas compartilham o mesmo nome.
- ▶ Regras para métodos sobrecarregados:
 - ▶ A lista de parâmetros deve ser diferente.
 - ▶ O tipo retornado pode ser diferente.

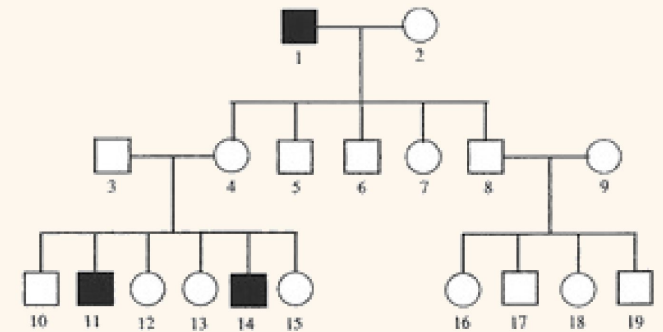


Exemplo 1 – Método println

```
public class ExemploSobrecargaPrintln {  
    public static void main(String[] args) {  
        int numero = 10;  
        String texto = "Olá, mundo!";  
        boolean logico = true;  
  
        // Usando diferentes versões do método println  
        System.out.println(numero); // Imprime o valor inteiro  
        System.out.println(texto); // Imprime o valor da string  
        System.out.println(logico); // Imprime o valor booleano  
    }  
}
```

Exemplo 2

- ▶ Uma classe matemática poderia ter os seguintes métodos, que retornam o maior entre dois parâmetros passados para ela:
 - ▶ `public static int maior(int a, int b);`
 - ▶ `public static long maior(long a, long b);`
 - ▶ `public static double maior(double a, double b);`



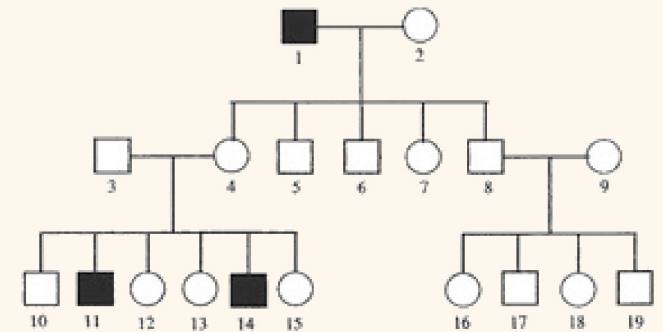
Sobrecarga de métodos

- ▶ Qual método será chamado?
 - ▶ O JAVA resolve que método chamar em tempo de compilação
 - ▶ O método chamado é aquele que apresentar tipos de parâmetros iguais ou mais “próximos” dos tipos dos valores passados na chamada



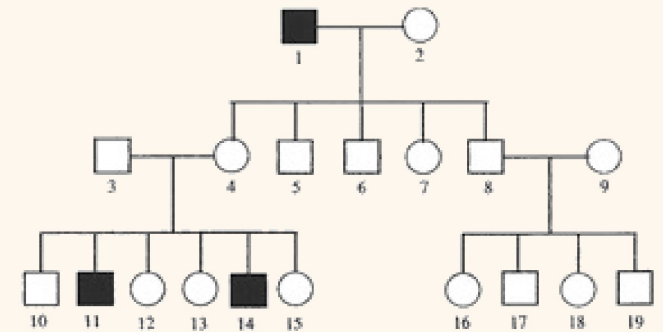
Sobrecarga de métodos

- ▶ No exemplo anterior, se não existisse sobrecarga de métodos, teríamos que renomear os métodos como por ex:
 - ▶ `public static int maior_int(int a, int b);`
 - ▶ `public static long maior_long(long a, long b);`
 - ▶ `public static double maior_double(double a, double b);`
- ▶ Outras linguagens resolvem essa questão com o uso de parâmetros opcionais:
 - ▶ Solução ruim: Um mesmo método pode acabar tendo uma quantidade muito grande de parâmetros.



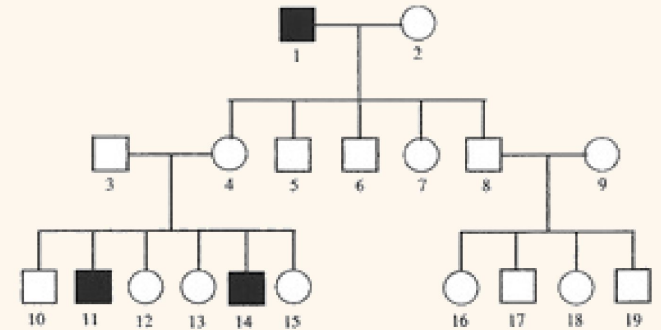
Sobrecarga de métodos

- ▶ Ainda no exemplo anterior, sem a sobrecarga, o método maior perderia o seu significado, o qual independe dos parâmetros utilizados.
- ▶ Isso significaria:
 - ▶ Mais trabalho para o programador, que teria que lidar com vários nomes diferentes para métodos que veiculam o mesmo conceito.
 - ▶ Perda do comportamento polimórfico.



Sobrecarga de métodos

- ▶ Útil quando:
 - ▶ Temos métodos cujo significado independe de seus parâmetros.
 - ▶ O conceito veiculado pelo método não muda quando alteramos o tipo dos parâmetros.
 - ▶ Desejamos ter várias opções na hora de construir um objeto (vários construtores).



Sobrecarga de construtores

- ▶ Uma classe pode possuir vários construtores.
- ▶ Vantagens:
 - ▶ Flexibilidade na hora de construir um objeto, com diferentes opções de construtores.
 - ▶ Reuso: Um construtor pode chamar outro construtor de uma mesma classe usando a palavra reservada “this” (quando isto acontece, o construtor não precisa chamar o “super”, isso ficará a cargo do outro construtor chamado).



Exemplo

```
class Pessoa {  
    private String nome;  
    private int idade;  
    public Pessoa() {  
        this("Desconhecido", 0);  
    }  
    public Pessoa(String nome) {  
        this(nome, 0);  
    }  
    public Pessoa(String nome, int idade) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
    public void exibirDados() {  
        System.out.println("Nome: " + nome + ", Idade: " + idade);  
    } }  
}
```

```
public class TestePolimorfismo {  
  
    public static void main(String[] args) {  
        Pessoa p1 = new Pessoa();  
        Pessoa p2 = new Pessoa("Ana");  
        Pessoa p3 = new Pessoa("Carlos", 30);  
  
        p1.exibirDados(); // Nome: Desconhecido, Idade: 0  
        p2.exibirDados(); // Nome: Ana, Idade: 0  
        p3.exibirDados(); // Nome: Carlos, Idade: 30  
    }  
}
```

Exercício

- ▶ Existem diferentes formas de calcular a área de um triângulo.
- ▶ Construa uma classe Triângulo. A classe deverá possuir um método getArea que será sobrecarregado, para suportar as diferentes formas de cálculo da área.
- ▶ As fórmulas podem ser vistas ao lado.

1. Base e altura

Fórmula: $A = (\text{base} \times \text{altura}) / 2$

2. Três lados (fórmula de Heron)

Fórmula de Heron:

$$s = \frac{a+b+c}{2}$$

$$A = \sqrt{s(s-a)(s-b)(s-c)}$$

3. Dois lados e ângulo entre eles (em graus)

Fórmula:

$$A = \frac{ab \sin(C)}{2}$$

Solução

```
class Triangulo {  
    public double getArea(double base, double altura) {  
        return (base * altura) / 2;  
    }  
    // Área com os três lados (fórmula de Heron)  
    public double getArea(double a, double b, double c) {  
        double s = (a + b + c) / 2;  
        return Math.sqrt(s * (s - a) * (s - b) * (s - c));  
    }  
    // Área com dois lados e o ângulo entre eles (em graus), o quarto parâmetro é só para  
    // diferenciar da lista de parâmetros do método anterior.  
    public double getArea(double a, double b, double anguloEmGraus, boolean usaAngulo) {  
        double anguloEmRadianos = Math.toRadians(anguloEmGraus);  
        return (a * b * Math.sin(anguloEmRadianos)) / 2;  
    }  
}
```

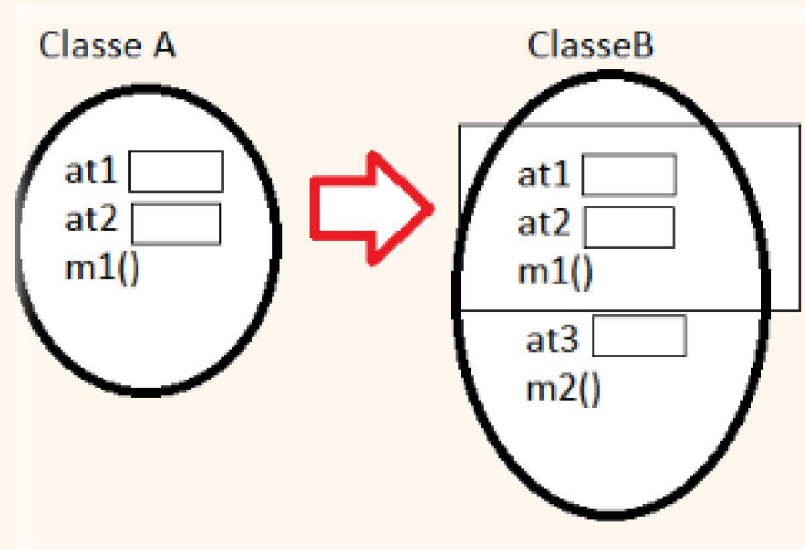
Princípio da substituição

- ▶ **Princípio da substituição: é a possibilidade de referenciar objetos de subtipos com variáveis de supertipos.**
 - ▶ Uma variável pode ter um TIPO e o objeto para o qual ela aponta pode ter outro TIPO diferente (desde que esse objeto herde direta ou indiretamente do tipo da variável).
 - ▶ Na linguagem Java, essa conversão é feita de forma implícita.
 - ▶ Uma variável de tipo não primitivo define como o programador “enxerga” o objeto para o qual ela aponta
 - ▶ Sua existência acarreta na necessidade do uso responsável da herança por parte do programador.

Princípio da substituição

- ▶ Subclasses herdam a IDENTIDADE das suas superclasses diretas e indiretas, até o topo da hierarquia.
- ▶ Isto significa que...se B herda de A (direta ou indiretamente), **B é A**
- ▶ De fato, se olharmos para um objeto do tipo B podemos enxergar em parte dele um objeto do tipo A, ou seja, todos os atributos e métodos herdados de A.

```
class A {  
    at1  
    at2  
    m1()  
}  
class B extends A {  
    at3  
    m2()  
}
```



Conversão implícita

- ▶ Nome dado ao processo de atribuirmos um objeto de uma subclasse a uma variável do tipo de uma superclasse sem necessidade de realizarmos um cast (conversão).
 - ▶ Pode se dar por atribuição entre variáveis ou entre variável e um comando “new”.
-

Exemplo

```
class Animal {
    public void interagir() {    System.out.println("O animal interage");    }
}
class Cachorro extends Animal {
    public void interagir() {    System.out.println("Au au au!");    }
    public void coçar() {    System.out.println("Cachorro se coçando.");    }

}
class Gato extends Animal {
    public void interagir() {    System.out.println("Miau....");    }
}
public class TestePolimorfismo {
    public static void main(String[] args) {
        Animal a1 = new Cachorro();    // Conversão implícita
        Gato g1 = new Gato();
        Animal a2 = g1;    // Conversão implícita
        a1.interagir();    // Saída: "Au au au!"
        a2.interagir();    // Saída: "Miau...."
    }
}
```

Conversão explícita

- ▶ Ocorre quando desejamos converter de uma superclasse para uma subclasse.
- ▶ Nesse caso, um cast (indicando o nome da subclasse entre parênteses) é necessário, e deve-se ter certeza de que o objeto apontado pela variável realmente é da subclasse, do contrário uma exceção será lançada. Ex:

```
Animal animal = new Cachorro(); // conversão implícita
```

```
Cachorro c = (Cachorro) animal; // conversão explícita realizada corretamente
```

```
c.coçar();
```

```
Animal animal2 = new Gato();
```

```
Cachorro c2 = (Cachorro) animal2; // Erro ClassCastException: class Gato cannot be cast to  
// class Cachorro (tempo de execução)
```

Operador instanceof

- Usado para verificar se um objeto é de um tipo específico (ou uma subclasse dele). É útil para evitar exceções.

```
public static void verificarAnimal(Animal a) {  
    if (a instanceof Cachorro) {  
        Cachorro c = (Cachorro) a; // conversão segura pois foi verificada  
        c.interagir();  
    }  
}
```

Generics

- ▶ Permite que Classes e interfaces possam ser usados como parâmetros quando formos definir novas classes, interfaces e métodos.
- ▶ Seu uso é especialmente importante quando se usa coleções, pois evita os problemas / necessidade de realizar casts e comparações com **instanceof**, vistos no slide anterior.

Tipo genérico

- ▶ Um tipo genérico é uma classe ou interface genérica que recebe tipos como parâmetro. Uma classe genérica pode ser definida da seguinte forma:
- ▶ Nome da classe<T1, T2, ..., Tn> { /* ... */ }
- ▶ O parâmetro do tipo é especificado após o nome da classe, dentro dos sinais de desigualdade(<>), especificando os tipos T1, T2, ..., and Tn.
- ▶ Vamos ver um exemplo.

Exemplo

- Considere uma classe Repositorio que armazena um único objeto (tipo Object):

```
class Repositorio {  
    private Object objeto;  
  
    public void set(Object objeto) {  
        this.objeto = objeto;  
    }  
    public Object get() {  
        return objeto;  
    }  
}
```

Exemplo com generics

- ▶ Para converter a classe Repositório para Generics, vamos adicionar uma declaração de tipo genérico alterando "public class Repositorio" para "public class Repositorio<T>".
- ▶ A variável de tipo T poderá então ser usada em qualquer local na classe.
- ▶ Todos os tipos não primitivos poderão ser usados no lugar de T: classes, interfaces, arrays...
- ▶ Substituiremos na classe repositório todas as ocorrências de Object por T.

Exemplo com generics

- ▶ Classe repositório com generics:

```
class Repositorio<T> {  
    // T: Tipo  
    private T t;  
  
    public void set(T t) {  
        this.t = t;  
    }  
    public T get() {  
        return t;  
    }  
}
```

- ▶ Classe original:

```
class Repositorio {  
    // armazena qualquer objeto  
    private Object objeto;  
  
    public void set(Object objeto) {  
        this.objeto = objeto;  
    }  
    public Object get() {  
        return objeto;  
    }  
}
```

Utilizando um tipo genérico

- ▶ Para utilizar o repositório genérico, devemos passar um tipo ao repositório e instanciar o objeto:

```
Repositorio<String> stringRep = new Repositorio<String>();
```

- ▶ Também é possível instanciar da seguinte forma, sem o tipo próximo ao parênteses:

```
Repositorio<String> stringRep = new Repositorio<>();
```

Utilizando um tipo genérico

```
public static void main(String[] args) {  
  
    Repositorio<String> stringRep = new Repositorio <String>();  
  
    stringRep.set(new String("Teste armazenamento generics"));  
  
    System.out.printf("String armazenada :%s\n", stringRep.get());  
}
```

Exemplo usando uma coleção sem generics

- ▶ ArrayList é uma classe que permite armazenar objetos, definindo uma lista de objetos que pode crescer ou diminuir.
- ▶ Seus métodos recebem um parâmetro do tipo object, então qualquer objeto Java pode ser passado de forma implícita para eles.
- ▶ A conversão implícita, sem generics, requer entretanto o uso de casts ao recuperar objetos.

```
List list = new ArrayList();
```

```
// c2 é uma instância da classe Cliente
```

```
list.add(c2);
```

```
..... // em outro lugar do código, recuperamos um cliente.
```

```
Cliente c = (Cliente) list.get(0);
```

Exemplo com uso de Generics

- ▶ O exemplo anterior, escrito agora com generics, evita a necessidade de realizar casts.

```
List<Cliente> list = new ArrayList<Cliente>();
```

```
// c2 é uma instância da classe Cliente
```

```
list.add(c2);
```

```
Cliente c = list.get(0); // sem precisar fazer cast
```

Exemplo com uso de coleções

```
import java.util.*;

public class Zoologico {
    public static void main(String[] args) {
        List<Animal> animais = new ArrayList<>();
        animais.add(new Cachorro());
        animais.add(new Gato());

        for (Animal a : animais) {
            a.interagir(); // Cada objeto chama sua própria implementação
        }
    }
}
```

Classes abstratas

- ▶ Tipo de classe que pode conter tanto métodos abstratos quanto métodos convencionais.
 - ▶ Não podem ser instanciadas. Sua utilização exige o uso de subclasses que irão implementar os métodos abstratos;
 - ▶ São criadas para serem estendidas.
-

Métodos abstratos

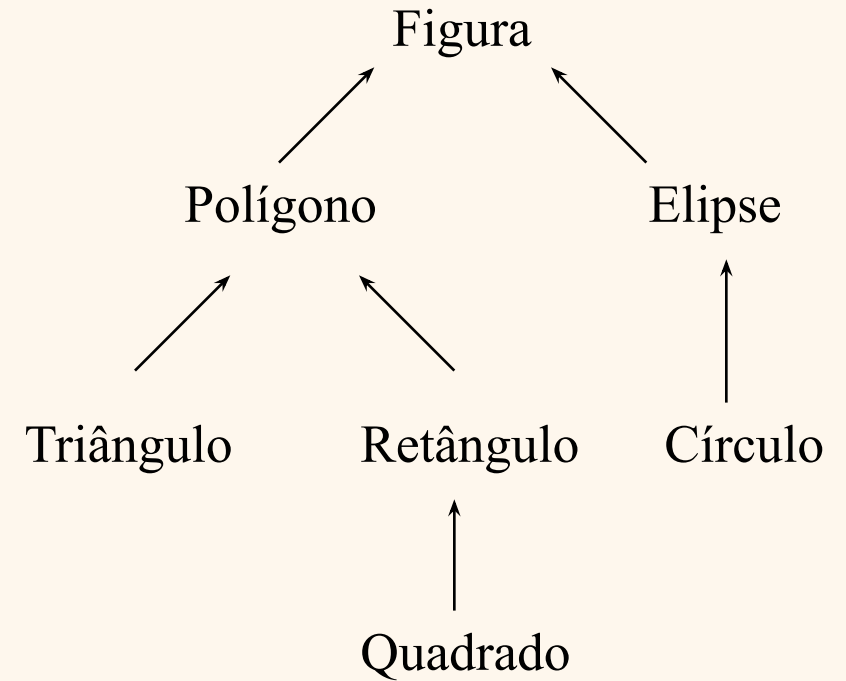
- ▶ Métodos que não possuem corpo (implementação). Contém apenas a declaração.
 - ▶ Sua presença torna a classe automaticamente abstrata.
 - ▶ Devem ser implementados nas subclasses da classe abstrata que os definiu.
-

Classes abstratas

- ▶ Uma classe deve ser declarada abstrata se:
 - ▶ A classe possui um ou mais métodos abstratos.
 - ▶ A classe herda um ou mais métodos abstratos, os quais não são implementados.
 - ▶ A classe declara que implementa uma determinada interface mas não fornece implementação para todos os métodos desta interface.
-

Exemplo

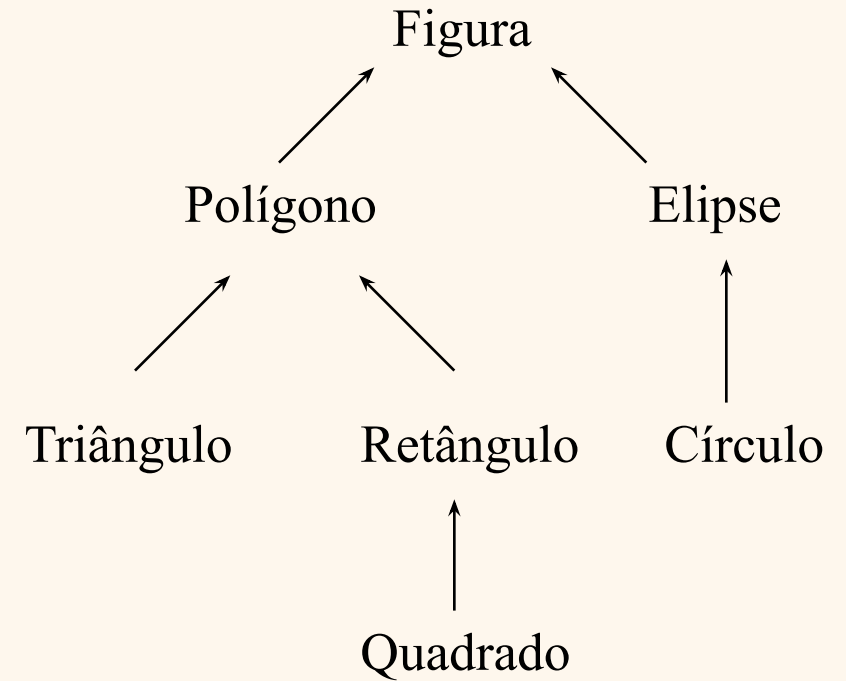
- ▶ Na hierarquia de classes ao lado, a classe Figura define uma figura genérica, que contém um método desenhar.
- ▶ É complicado definir um método desenhar para uma figura genérica, que pode ter qualquer forma; Definiremos este método então como abstrato, devendo ser implementado em cada subclasse de Figura.



Exemplo

- ▶ Código da classe Figura:

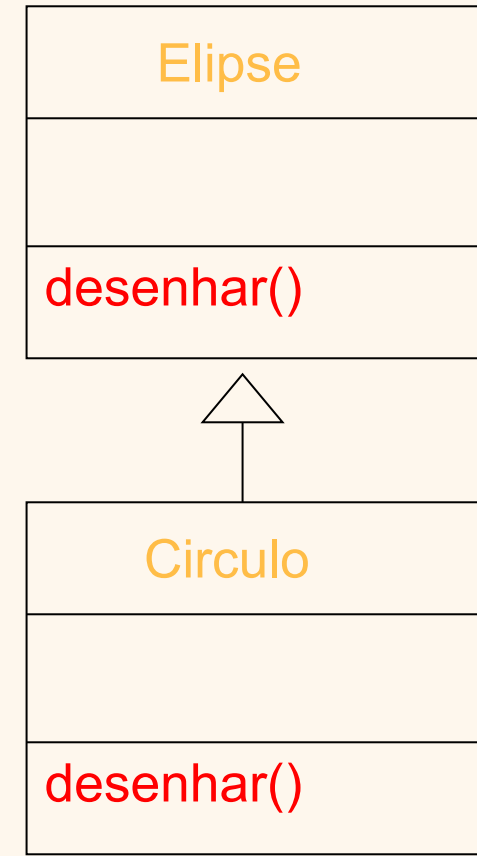
```
abstract class Figura {  
  
    // subclasses de figura devem fornecer uma  
    // implementação para esse método.  
    public abstract void desenhar( );  
}
```



Exemplo

```
class Elipse extends Figura {  
    public void desenhar () {  
        // código para desenhar uma elipse  
    }  
}
```

```
class Circulo extends Elipse {  
    public void desenhar () {  
        // código para desenhar um círculo  
    }  
}
```



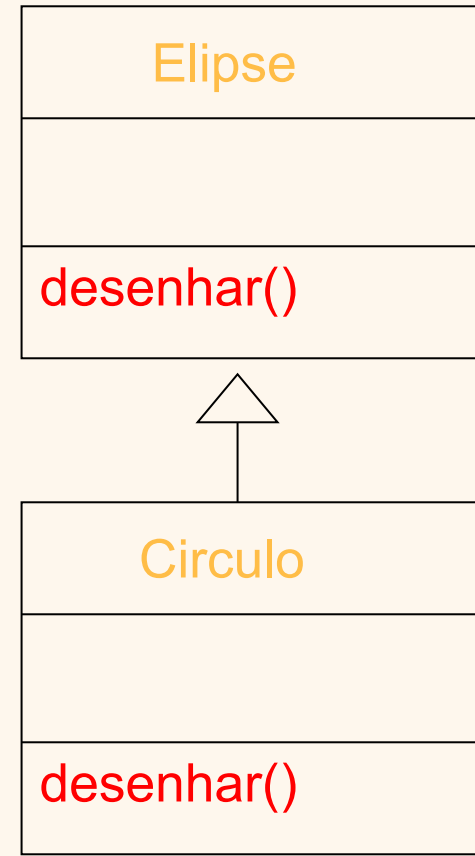
Exemplo

```
Figura figura1 = new Elipse();
```

```
Figura figura2 = new Círculo();
```

```
figura1.desenhar();
```

```
figura2.desenhar();
```



Classes abstratas “puras”

- ▶ Possuem apenas métodos abstratos;
 - ▶ Os atributos, quando presentes, se resumem a constantes estáticas;
 - ▶ Podem ser definidas como interfaces, para maior flexibilidade no uso.
-

Interfaces

- ▶ Tipo de classe que possui apenas métodos abstratos; Os atributos, se existirem, devem ser public, static e final.
 - ▶ Não podem ser instanciadas. Sua utilização exige o uso de uma classe que implemente a interface;
 - ▶ Classes que desejam implementar uma interface utilizam a palavra reservada **implements**.
-

Interfaces

- ▶ Funcionam como um “contrato”: Uma classe que implementa uma interface deve implementar todos os métodos dessa interface;
 - ▶ Ao implementar uma interface, a classe mantém o direito de herdar de outra classe.
 - ▶ Uma classe pode implementar diversas interfaces, incrementando o polimorfismo.
-

Exemplo

```
interface MinhaInterface {  
    public void meuMetodoAbstrato ();  
}
```

```
class MinhaClasse implements MinhaInterface {  
    public void meuMetodoAbstrato() {  
        System.out.println("Método implementado!");  
    }  
}
```

Vantagens do uso de Interfaces

- ▶ Aumentam a reusabilidade do software.
 - ▶ Reduzem o esforço de evolução da aplicação.
 - ▶ Facilitam a criação de arquiteturas baseadas em componentes.
-



**Slides adaptados a partir
do material produzido pelo
prof. Eduardo Calábria**

egc2@cesar.school





**MAURÍCIO DA
MOTTA BRAGA**



mmb2@cesar.school

